457_a1q1

2026-01-28

Qustion 1

Script

```
# Load datasets (assuming they are in the working directory)
train_data <- read.csv("train_Q1A1.csv")
test_data <- read.csv("test_Q1A1.csv")

# Extract columns
x_train <- train_data[[1]]
y_train <- train_data[[2]]
x_test <- test_data[[1]]
y_test <- test_data[[2]]

# Kernel Regression Function
kernel_reg_predict <- function(x, y, test_x, h) {
  n <- length(x)
  predictions <- sapply(test_x, function(xt) {
    # Identify indices within distance h
    distances <- abs(xt - x)
    in_range <- distances <= h

    # Check if any points are within distance h
    num_points <- sum(in_range)

    if (num_points > 0) {
      # Return the average of the responses within the window
      return(mean(y[in_range]))
    } else {
      # Fallback convention: global mean if no points are within h
      return(mean(y))
    }
  })
  return(predictions)
}

# Setup bandwidth values and error storage
h_values <- c(0.01, 0.03, 0.05, 0.07, 0.09, 0.11)
results <- data.frame(h = h_values, train_mse = NA, test_mse = NA)

# Calculate MSE for each h
for (i in 1:length(h_values)) {
```

```r
  h <- h_values[i]

  # Predictions for Training and Testing sets
  pred_train <- kernel_reg_predict(x_train, y_train, x_train, h)
  pred_test  <- kernel_reg_predict(x_train, y_train, x_test, h)

  # Calculate Mean Square Errors
  results$train_mse[i] <- mean((y_train - pred_train)^2)
  results$test_mse[i]  <- mean((y_test - pred_test)^2)
}

print(results)
```

```
##      h train_mse  test_mse
## 1 0.01 0.5577589 0.6289642
## 2 0.03 0.6232425 0.5419921
## 3 0.05 0.6577334 0.5328887
## 4 0.07 0.6593200 0.5265414
## 5 0.09 0.6821328 0.5284520
## 6 0.11 0.6966682 0.5379791
```

```r
# Find the h with the smallest testing error
results$h[which.min(results$test_mse)]
```

```
## [1] 0.07
```

Observations

This is a demonstration of the Bias-Variance Tradeoff. As the bandwidth h increases, we can see that the training MSE increases steadily.

- When h is small, the model only looks at very close neighbors to make a prediction. This means the model is strict (flexible) and may overfit to the data points nearby, resulting in low bias but high variance.
- When h is large, the model considers a much larger set of neighboring points, this smooths out the prediction. This leads to underfitting, because we captures too much variety and outliers, causing the fitted line to become too generalized. This also means we have higher bias and lower variance.

The perfect h

We can see that h = 0.07 achieves the lowest test error. When h is too small the model overfits training data, while when h is too large the model generalizes (underfits true distribution) too much. Testing MSE being shown in this U shape is what we'd typically expect. We want to pick h such that the testing MSE is the minimum of that parabolic shaped function, which is 0.07. This point minimizes the Bias-Variance Tradeoff.

Question 2

```r
set.seed(1)
nfold <- 10

# Assign each observation to one of the 10 folds randomly
infold <- sample(rep(1:nfold, length.out = length(x_train)))

h_candidates <- c(0.01, 0.03, 0.05, 0.07, 0.09, 0.11)
# Matrix to store MSE: Rows = h values, Columns = folds
errorMatrix <- matrix(NA, nrow = length(h_candidates), ncol = nfold)

# Perform 10-Fold CV
for (l in 1:nfold) {
  # Split data into training fold and validation fold
  train_x_cv <- x_train[infold != l]
  train_y_cv <- y_train[infold != l]
  valid_x_cv <- x_train[infold == l]
  valid_y_cv <- y_train[infold == l]

  for (i in 1:length(h_candidates)) {
    h <- h_candidates[i]

    # Get predictions for the validation fold using the other 9 folds
    preds <- kernel_reg_predict(train_x_cv, train_y_cv, valid_x_cv, h)

    # Calculate and store MSE for this fold
    errorMatrix[i, l] <- mean((valid_y_cv - preds)^2)
  }
}

# Calculate Average Cross Validation Error per h
cv_errors <- apply(errorMatrix, 1, mean)

# Show Results
results_cv <- data.frame(h = h_candidates, Average_CV_MSE = cv_errors)
print(results_cv)
```

```
##      h Average_CV_MSE
## 1 0.01      0.8029360
## 2 0.03      0.6946407
## 3 0.05      0.7025000
## 4 0.07      0.6847648
## 5 0.09      0.6999980
## 6 0.11      0.7108376
```

```r
# Find the best h
h_candidates[which.min(cv_errors)]
```

```
## [1] 0.07
```

Observations

Once again, h=0.07 produced the smallest cross-validation error, which is what we'd expect based on our previous attempt at finding the optimal h.

Q3. $A \in \mathbb{R}^{d \times d}$ $\quad A = U \Lambda U^T \quad U \in \mathbb{R}^{d \times d}$, orthonormal, $\Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_d)$

a) Show if $\theta = (\theta_1, \dots, \theta_d)^T \in \mathbb{R}^d$ has length 1, then $\lambda_d \le \sum_{i=1}^{d} \theta_i^2 \lambda_i \le \lambda_1$

$\lambda_i \ge \lambda_d$ ← Given

$\theta_i^2 \lambda_i \ge \theta_i^2 \lambda_d$ ← multiply both sides by θ_i^2, which always ≥ 0

$\sum_{i=1}^{d} \theta_i^2 \lambda_i \ge \sum_{i=1}^{d} \theta_i^2 \lambda_d$ ← since left side always ≥ right side summation over all is should stay true.

$\sum_{i=1}^{d} \theta_i^2 \lambda_i \ge \lambda_d \underbrace{\sum_{i=1}^{d} \theta_i^2}_{=1}$ ← isolate constant

$\sum_{i=1}^{d} \theta_i^2 \lambda_i \ge \lambda_d$ $\longrightarrow$ $\therefore \lambda_d \le \sum_{i=1}^{d} \theta_i^2 \lambda_i \le \lambda_1$

Conversely $\lambda_1 \ge \lambda_i$

$\theta_i^2 \lambda_1 \ge \theta_i^2 \lambda_i$

$\sum_{i=1}^{d} \theta_i^2 \lambda_1 \ge \sum_{i=1}^{d} \theta_i^2 \lambda_i$

$\lambda_1 \cdot 1 \ge \sum_{i=1}^{d} \theta_i^2 \lambda_i$

Implication: think of θ_i as a weight

the weighted average of eigenvalues must be between λ_1 (largest eig, all weight allocated to λ_1) and λ_d (smallest eig, all weight allocated to λ_d)

Let $\theta = (0, 0, \dots, 0, 1)^T$

then $\theta^2 = (0, 0, \dots, 1)^T$

meaning $\theta_{i \ne d}^2 = 0$ and $\theta_d^2 = 1$

Then $\sum_{i=1}^{d} \theta_i^2 \lambda_i = 0 \cdot \lambda_i + \dots + 0 \cdot \lambda_{d-1} + 1 \cdot \lambda_d = \lambda_d$

Then $\lambda_d = \sum_{i=1}^{d} \theta_i^2 \lambda_i \le \lambda_1$

Let $\theta = (1, 0, \dots, 0, 0)^T$

$\theta^2 = (1, 0, \dots, 0, 0)$

$\theta_{i \ne 1}^2 = 0 \quad \theta_1^2 = 1$

Then $\sum_{i=1}^{d} \theta_i^2 \lambda_i = 1 \cdot \lambda_1 + \dots + 0 \cdot \lambda_d$

Then $\lambda_1 = \sum_{i=1}^{d} \theta_i^2 \lambda_i \ge \lambda_d$

Thus we have proved that the 2 ends of the inequality is bounded.

b) Show $v^T A v = \sum_{i=1}^{d} \theta_i^2 \lambda_i$ given $v = U\theta$

$$v^T A v = (U\theta)^T \cdot U \Lambda U^T \cdot (U\theta) \quad \leftarrow \text{sub in}$$

$$= \theta^T U^T \cdot U \Lambda U^T U \theta \quad \leftarrow (AB)^T = B^T A^T$$

$$= \theta^T \Lambda \theta \quad \leftarrow U^T U = I \text{ because } u_i \cdot u_j = 0 \text{ for } i \neq j, \; u_i \cdot u_i = 1$$

$$= (\theta_1, \dots, \theta_d) \cdot \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_d \end{bmatrix} \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_d \end{bmatrix}$$

$$= \sum_{i=1}^{d} \lambda_i \theta_i \cdot \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_d \end{bmatrix} = \sum_{i=1}^{d} \theta_i^2 \lambda_i$$

c) Since $v = U\theta$

$$\|\vec{v}\|^2 = \|U\theta\|^2$$

$$= \theta^T U^T \cdot U\theta \quad \text{but } U^T U = I$$

$$= \theta^T \theta$$

$$= \|\theta\|^2 \quad \text{so } \|v\| = 1 \iff \|\theta\| = 1$$

from b) $v^T A v = \sum_{i=1}^{d} \theta_i^2 \lambda_i$ from a) $\lambda_d \leq \sum \theta_i^2 \lambda_i \leq \lambda_1$ if $\|\theta\| = 1$

Then we have demonstrated that λ_d is the smallest possible value of $v^T A v$ using a. and so is λ_1 the largest value.

If A is positive definite

$\rightarrow$ $v^T A v > 0$ for all possible $\vec{v}$.

$\rightarrow$ we proved $\lambda_d = \min_{v \in \mathbb{R}^d : \|v\| = 1} v^T A v$

$\rightarrow$ then $\lambda_d > 0$

$\rightarrow$ since $\lambda_d \leq \lambda_i \leq \lambda_1$

then all eigenvalues are positive.

If all eigenvalues are positive

$\rightarrow$ $v^T A v = \sum_{i=1}^{d} \theta_i^2 \lambda_i$ from b)

$\rightarrow$ $\sum_{i=1}^{d} \theta_i^2 \lambda_i \geq \lambda_d$

$\rightarrow$ $v^T A v \geq \lambda_d$

$\rightarrow$ given all $\lambda_i > 0$, $\lambda_d > 0$

$\rightarrow$ $v^T A v > 0$ $\rightarrow$ A is positive definite.

4. MVT: $\exists c\ (x,y)$ s.t.: $\dfrac{\phi(y)-\phi(x)}{y-x} = \phi'(c)$

a) Case 1: $y > x$ then $x < c < y$

since $\phi'(x)$ is non-decreasing, then $\phi'(c) \geq \phi'(x)$

then $\dfrac{\phi(y)-\phi(x)}{y-x} \geq \phi'(x)$ since $y > x$, $y-x > 0$

$$\phi(y)-\phi(x) \geq \phi'(x)(y-x)$$

$$\phi(y) \geq \phi'(x)(y-x) + \phi(x)$$

Case 2: $y < x$ they $y < c < x$

similarly $\phi'(c) \leq \phi'(x)$

$\dfrac{\phi(y)-\phi(x)}{y-x} \leq \phi'(x)$ $y-x < 0$ flip inequality sign.

$$\phi(y)-\phi(x) \geq \phi'(x)(y-x)$$

$$\phi(y) \geq \phi'(x)(y-x) + \phi(x)$$

Case: $y = x$, denominator is 0, so inequality is $0 \geq 0$ which is trivial.

b)

$$\begin{array}{ll} 2M|u| - M^2 & \text{if } |u| > M \\ 2Mu - M^2 & \text{if } u > M \\ -2Mu - M^2 & \text{if } -u > M \end{array} \Rightarrow \phi_M'(u) = \begin{cases} 2u & \text{if } |u| \leq M \\ 2M & \text{if } u > M \\ -2M & \text{if } u < -M \end{cases}$$

we can see $u \in (-\infty, -M)$, derivative is $-2M$ (constant)

$u \in [-M, M)$, derivative is $2u$ (increasing) also continuous.

$u \in [M, \infty)$, derivative is $2M$ (constant)

$\phi'(u)$ is a non-decreasing continuous function $\rightarrow$ by part a) $\rightarrow$ $\phi(u)$ must be a convex function.

Question 5

```r
library(CVXR)
```

```
##
## Attaching package: 'CVXR'

## The following object is masked from 'package:stats':
##
##     power
```

```r
data <- read.csv("dat_Q5A1.csv", header = TRUE)

y <- as.matrix(data[, 1])
X <- as.matrix(data[, 2:4])

n <- nrow(X)
d <- ncol(X)
M <- 0.1

beta <- Variable(d)

# Solve the optimization problem
objective <- sum(CVXR::huber(y - X %*% beta, M))
problem <- Problem(Minimize(objective))
result <- solve(problem)

beta_hat <- result$getValue(beta)
print(beta_hat)
```

```
##             [,1]
## [1,]  0.98397821
## [2,] -0.79800742
## [3,]  0.02582864
```